

Status_Summary

Feature Status Summary

Revision: 8 **Last modified:** 2026-06-16T09:52:00Z **Authority:** constitution §11.4.56 (Status_Summary two-audience companion to §11.4.153 Status.md), §11.4.44 (revision header), §11.4.65 + §11.4.153 (HTML+PDF+DOCX export). **Companion of:** docs/features/Status.md.

Page 1 — For the team (non-developer, plain language)

What this project is

A self-hosted toolkit that downloads videos from YouTube and many other sites (Instagram, Facebook, X, TikTok, Bilibili, and more). It runs as a small set of containers you start with simple commands. There are three ways to use it:

- **Бoбa landing page** — a friendly welcome page that walks you through signing in to your sites and uploading your cookies so login-protected videos can be downloaded.
- **Dashboard** — a modern web app where you paste a link, pick quality, and watch your downloads progress, see history, and manage cookies.
- **Command line** (`./download`) — for power users who prefer the terminal.

An optional VPN tunnel can route all downloads privately, and an auto-update service keeps the container images fresh.

What works today

- Downloading videos through the dashboard, the classic MeTube interface, or the command line.
- The cookie sign-in flow (upload your `cookies.txt`, see how fresh it is per site).
- The download queue with live status (waiting, preparing, downloading, finishing, done, error, cancelled) and a history list that even remembers cancelled items.
- Starting, stopping, restarting, and checking the health of the whole system with one command each.
- An optional VPN mode and automatic image updates.

What is recorded so far

- **Nine screens/transcripts are now video-confirmed** by a real capture of the genuinely-running system: the Dashboard's four tabs (Download, Queue, History, Cookies — all four examined individually), the Бoбa landing page, the classic MeTube interface, the post-processing status page, the `./status` health command, and the `./download` command. Each was examined and confirmed working — no blank, frozen, or error screens. The History tab shows 46 real downloads; the Cookies tab shows the real per-site cookie status (YouTube and X cookies have expired, which is why login-protected downloads need fresh cookies). These confirm the screens *look right and respond*.
- **One full end-to-end flow is now confirmed too:** downloading a video all the way to a web-ready copy. We watched the live pipeline finish one more item (its "done" count went up by one) and verified the newly-produced web-ready file is a genuine, playable video (correct H.264 format, 1920×1080, ~2 hours long, fast-start enabled). The two flows still to record are MP3 audio extraction (no audio job has run recently, so no MP3 file exists to confirm yet) and the cookie upload start-to-finish — the latter needs the operator's own fresh login cookies, and uploading test cookies would overwrite the real ones, so it waits on the operator.
- Every other feature still reads **"PENDING — not yet recorded."** This is the honest status, not a placeholder.

What is now built (was "planned")

- The **media post-processing** capability is **now built and shipped** (version `ytdlp-1.4.0`): it makes web-ready video copies (6+ already on disk), can pull out MP3 audio, and reports live pipeline progress through a status page.
- Still genuinely **not built**: a dashboard screen that shows pipeline progress visually, and a "resume an interrupted download" feature.
- **The automated tests now have a captured, evidence-backed green run — 189 tests this session, zero real failures:** 69/69 post-processor unit/integration, 22/22 live smoke, 18/18 API-contract, **63/63 dashboard (Angular) UI-component tests**, and 17/17 pure-shell unit tests — all saved under `qa-results/`. So the post-processor AND the dashboard components are confirmed PASS, not "pending". The remaining "PENDING_FORENSICS" verdicts are for tests that start/stop containers, which were deliberately not run so the live released system stays up.
- **Local image analysis was proven to work on this machine:** a small vision model (Qwen2.5-VL-3B via `mlx`, using the Mac's GPU, no cloud) described a real dashboard screenshot correctly in ~20 seconds. It's a useful helper but occasionally invents small plausible details, so the assistant's own reading of screenshots stays the primary check.

Team actions

- None blocking. The VPN mode needs the operator's own VPN credentials to run live, but the code path is in place.
 - Next milestone: record the end-to-end **flow** videos (download → web-ready, cookie upload) so those features move from render-confirmed to flow-confirmed.
-

Page 2 — For software engineers

File-path anchors

- **Services:** `docker-compose.yml` — `openvpn-yt-dlp`, `metube` / `metube-direct`, `landing-vpn` / `landing-no-vpn`, `yt-dlp-cli` / `yt-dlp-cli-vpn`, `dashboard`, `watchtower`, `media_postprocessor` (container `media-postprocessor`, `MP_PORT=8089`). Profiles: `vpn`, `no-vpn`, `vpn-cli`, `docker`.
- **Media post-processor (ytdlp-1.4.0):** `media_postprocessor/` — `config.py`, `jobs_db.py` (SQLite-WAL queue), `media_probe.py` (ffprobe), `transcoder.py` (`transcode_video()` → `webready-<base>.mp4`; `derive_mp3()` → `<base>.mp3 320 kbps`), `watcher.py`, `worker.py`, `service.py` (/postprocess/*). `nginx proxy dashboard/` `nginx.conf.template:116`.
- **Landing (Flask 'Бо́а'):** `landing/app.py` — routes `/`, `/app`, `/api/upload-cookies`, `/api/delete-cookies`, `/api/cookie-status`, `/api/aborted-history` (GET/POST/DELETE), `/api/profile-status`, `/health`, `/logo.png`, `/favicon.ico`, `/api/delete-download`. Helpers `_validate_cookie_file`, `_summarize_cookies_by_platform`, `_aborted_history_lock` (flock at `/config/aborted.json.lock`).
- **Dashboard (Angular 17 + nginx):** `dashboard/src/app/` — `app.routes.ts` ('' `download-form`, `queue`, `history`, `cookies`, `** not-found`); components `download-form/`, `queue/` (STATE_META: `pending`, `preparing`, `downloading`, `postprocessing`, `finished`, `error`, `aborted`), `history/` (`aborted-history` merge via `abortedToDownloadInfo`), `cookies/`, `navbar/` (`profile badge`), `error-boundary/`, `not-found/`; services `metube.service.ts` (typed API, no any), `error-interceptor.service.ts`. `nginx template dashboard/` `nginx.conf.template + dashboard/entrypoint.sh`.
- **CLI scripts (repo root, no .sh suffix):** `init`, `start`, `start_no_vpn`, `stop`, `status`, `download`, `check-vpn`, `restart`, `cleanup`; `prepare-release.sh`, `setup-auto-update`, `update-images`. Runtime detection via `lib/container-runtime.sh`.
- **Tests:** `tests/test-*.sh` (`unit`, `integration`, `integration-realhttp`, `scenarios`, `errors`, `dashboard`, `dashboard-operations`, `aborted-history`, `cookie-validator`, `media-services`, `bulk-operations`, `add-all-platforms`, `add-download`, `vpn-compose`, `vpn-smoke`, `chaos`, `constitution-inheritance`); orchestrators `run-tests.sh`, `run-full-suite.sh`, `run-comprehensive-tests.sh`; `tests/e2e/` (Playwright: `dashboard`, `landing`, `cross-service`, `clear-history`, `delete-retry-cancel`, `cleanup-and-start`); `tests/benchmark/`, `tests/cookies/`, `tests/challenges/` (`metube-challenges.json + run-metube-challenges.sh`).
- **Challenges (HelixQA):** `challenges/scripts/` (`api-contract`, `container-restart-resilience`, `download-completes`, `form-reenables`, `host/user OOM+suspend`, `landing-cookie-upload`, `memory-limits`, `no-vpn-direct`, `queue-lifecycle`, `queue-polling-survives-error`, `retry-immediately-visible`, `run_all_challenges.sh`); `Challenges/ submodule (framework + p1-f06..f18 CLI-agent feature dirs)`.

Status vocabulary (§11.4.45 closed set)

PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED.

- Implemented-and-wired features: **PENDING_FORENSICS** — code present + test files exist, but no clean-baseline full-suite run with captured runtime evidence is on record in this revision (per §11.4.6 an unproven PASS is forbidden).
- Planned features (§7 of Status.md): **SKIP** — nothing to validate.
- OPERATOR-BLOCKED: none. VPN live path needs operator .ovpn credentials but the code path is implemented and compose-config-testable, so it is PENDING_FORENSICS, not blocked.

§-anchors

- §11.4.153 — per-feature Status + Status_Summary set with mandatory real-use video confirmation; adds DOCX to the export set for this class.
- §11.4.107 — no frozen/stale-frame video proof; videos (once recorded) must show live, advancing real-use content.
- §11.4.6 — no-guessing: every row reconciled against real code; no unproven PASS; no faked confirmation.
- §11.4.45 / §11.4.56 / §11.4.44 / §11.4.65 — status-doc maintenance, two-audience summary, revision header, multi-format export.

Video-confirmation status (load-bearing honest fact)

Per §11.4.153, EVERY user-visible confirmed claim must be backed by a recorded real-use capture. At Revision 7 the real PASSes span **TEN** ytdlp---prefixed captures — NINE render/transcript + ONE flow — each under /Volumes/T7/Downloads/Recordings/ and analyzed by Claude Opus 4.8 native multimodal (the strong-model path; local CPU vision rejected — see docs/research/vision-path/FINDINGS.md):

- ytdlp---dashboard---20260615T221723Z.png — Dashboard / (download-form + navbar).
- ytdlp---dashboard-queue---20260616T083018Z.png — Dashboard / queue (empty state).
- ytdlp---dashboard-history---20260616T083023Z.png — Dashboard / history (content, 46 items).
- ytdlp---dashboard-cookies---20260616T083029Z.png — Dashboard / cookies (content, per-platform).
- ytdlp---landing---20260616T074850Z.png — landing 'Боџа' UI render.
- ytdlp---metube---20260616T075608Z.png — MeTube UI render (real Completed table).
- ytdlp---postprocess---20260616T075648Z.png — postprocess status API render (live healthy JSON, 1 running / 25 done).
- ytdlp---status---20260616T075901Z.txt — ./status CLI transcript (all 5 HTTP health checks body-matched).
- ytdlp---download-cli---20260616T080936Z.txt — ./download CLI render (usage banner + detected podman runtime + wired podman exec yt-dlp-cli yt-dlp).
- ytdlp---landing-api---20260616T083612Z.txt — landing read-only-API sweep: seven §4 routes (/ , /health, /api/cookie-status, /api/profile-status, /api/aborted-history GET, /logo.png, /favicon.ico) each

HTTP 200/204 with real bodies (mutating POST/DELETE routes deliberately NOT auto-called — operator state).

- ytdlp---webready-flow---20260616T080633Z.txt —

download→webready FLOW: live pipeline done counter 25→26 across two real /api/postprocess/status snapshots + ffprobe-validated just-produced artifact (h264, 1920×1080, 7135s, faststart=YES, 3.34 GB). Backend pipeline (no UI) → §11.4.69/.107 sink-side evidence is the validated artifact, not a screen recording.

The first five are scope=render/transcript; the sixth is the first **flow** confirmation. The MP3-derivation and cookie-upload end-to-end FLOWS are NOT yet video-confirmed (§11.4.153 render-only ≠ flow-confirmed). Every other row carries Video-Confirmation = PENDING — not yet recorded. This is the truthful state (§11.4.2 / §11.4.5 / §11.4.6 / §11.4.107), not a stub.

Built-vs-planned (reconciled 2026-06-16, Rev 4)

The media_postprocessor sidecar, web-ready transcode, MP3 derivation, and post-process status API are **built and shipped** in tag ytdlp-1.4.0 — verified as FACT: real package media_postprocessor/ (config/jobs_db/media_probe/transcoder/watcher/worker/service), compose service media_postprocessor (MP_PORT=8089), nginx proxy dashboard/nginx.conf.template:116 /api/postprocess/ → media-postprocessor:8089, a live endpoint returning real JSON, and 6+ real webready-*.mp4 on disk. They are §7 of Status.md. The ONLY remaining Planned / SKIP rows (§8) are the dashboard pipeline-status UI (find dashboard/src/app -iname '*postprocess*' -o -iname '*pipeline*' → none) and a user-facing download-resume feature.